

# Notes on concurrency control in PostgreSQL

Maurizio Lenzerini

Data Management

A.Y. 2023-24

We remind the reader that in the SQL standard, the phenomena (called anomalies) which are prohibited at the various levels are:

- Dirty read

A transaction reads data written by a concurrent uncommitted transaction.

- Nonrepeatable read

A transaction re-reads data it has previously read and finds that data has been modified by another transaction (that committed since the initial read).

- Phantom read

A transaction re-executes a query returning a set of rows that satisfy a search condition and finds that the set of rows satisfying the condition has changed due to another recently committed transaction.

- Serialization anomaly

The result of successfully committing a group of transactions is inconsistent with all possible orderings of running those transactions one at a time.

There are four isolation levels defined by the SQL standard: *read uncommitted*, *read committed*, *repeatable read*, and *serializable*. The definition of those isolation levels says that

- *read committed* disallows *dirty read* anomalies,
- *repeatable read* disallows *dirty read* and *nonrepeatable read*, and
- *serializable* disallows all anomalies.

PostgreSQL doesn't implement *read uncommitted*, which allows *dirty reads*, and instead its default is *read committed*. PostgreSQL also disallows *phantom read* already in the *repeatable read* isolation level.

We remind the reader that a user interacts with the system through sessions. Every command (i.e., SELECT statement) given to the system that is not embraced into an explicit transaction within a session is considered a transaction. If a user wants to define an explicit transaction in a session, the BEGIN (or, equivalently, BEGIN TRANSACTION) command must be executed in the session. The transaction will be terminated by

- the COMMIT command, indicating a successful completion, or
- the ROLLBACK command, asking the system to undo all the operations, or
- the END command, that triggers the COMMIT command if the transaction can terminate successfully or the ROLLBACK command, otherwise.

When a transaction is aborted by the system, the ROLLBACK command is automatically executed and all the remaining actions are ignored until the END command or the explicit ROLLBACK command is issued.

If one uses the pgAdmin client, then every session corresponds to a tab. One can always leave a session A and switch to another session B by simply changing tab. In this case, session A remains active, and the user can always go back to it by accessing the corresponding tab. If the user closes the tab, then the corresponding session expires.

## 1. General considerations on PostgreSQL

As we said before, the default isolation level in PostgreSQL is READ COMMITTED, which is the first level. Thus, there is no pure READ UNCOMMITTED level in PostgreSQL. The following observations are important:

- The concurrency control strategy of PostgreSQL is a sort of multiversion control combined with (implicit and explicit) locking.
- Read actions are never blocked: they read the value written by the last committed transaction (recall that the lowest level for PostgreSQL is READ COMMITTED).
- Write actions are performed on a local store (snapshot), and their effects are transferred to the database at commit.
- Postgres keeps write (exclusive) locks until the end of the transaction, whereas read (shared) locks are released as soon as the execution of the SELECT operation is terminated.
- Explicit LOCK/UNLOCK commands are also allowed.
- Deadlock management is based on recognition.

In what follows, we refer to a database managed by PostgreSQL with the table “person” defined as follows:

```
CREATE TABLE person
(
    name character varying(40),
    age integer,
    salary integer,
    CONSTRAINT person_pkey PRIMARY KEY (name)
);
```

and with the following tuples:

| name    | age | salary |
|---------|-----|--------|
| Aldo    | 15  | 25     |
| Andrea  | 30  | 27     |
| Anna    | 20  | 50     |
| Filippo | 80  | 26     |
| Franco  | 20  | 60     |
| Gianna  | 40  | 50     |
| Luigi   | 40  | 50     |
| Luisa   | 87  | 75     |
| Maria   | 42  | 55     |
| Olga    | 41  | 30     |
| Sergio  | 35  | 85     |

To start our investigation, we now present three scenarios of usage of PostgreSQL, illustrating some basic features of PostgreSQL. The three scenarios illustrate the case of serial schedules, the use of locking and the deadlock management, respectively.

## 1.1 Scenario 1: serial schedules

Assume the initial situation of 80 as the value for the “age” attribute for Filippo. We will open three sessions, each one with one transaction.

### *Transaction 1*

We execute transaction 1, which coincides with the statement

```
select * from person where name = 'Filippo';
```

In the output we see 80 as the value of “age”, which is the value stored in the database before the beginning of the transaction. Since this transaction is just a single SQL (SELECT) statement, the transaction simply ends immediately with (implicit) COMMIT.

### *Transaction 2*

We execute transaction 2, which coincides with the statement

```
update person set age = 50 where name = 'Filippo';
```

and, since again this is just a single SQL statement, the corresponding transaction simply ends with (implicit) COMMIT.

### *Transaction 3*

We execute transaction 3, which coincides with the statement

```
select * from person where name = 'Filippo';
```

and in the output we see 50 as value of “age”, which is the value stored by second SQL statement (the update of transaction 2). Again, since this is just an SQL statement (SELECT), the corresponding transaction simply ends with (implicit) COMMIT.

Note that this behavior is obvious, since the schedule created by the above-described sequence of statements is serial, and the behavior of the system is the one that everybody expects.

## 1.2 Scenario 2: the use of implicit locking

Assume the initial situation of 26 as the value of the attribute “salary” for Filippo. We consider transaction 1 aiming at assigning 20 to the “salary” of Filippo, and transaction 2 aiming at assigning 30 to the “salary” of Filippo. Please, remember that PostgreSQL adopts READ COMMITTED as the default isolation level, which is also the lower isolation level.

### *Transaction 1*

We execute the following actions of transaction 1:

```
BEGIN;  
update person set salary = 20 where name = 'Filippo';
```

and we leave the session, thus suspending the transaction.

### *Transaction 2*

We execute the following actions of transaction 2:

```
BEGIN;  
update person set salary = 30 where name = 'Filippo';
```

and the system shows a message saying that it is waiting for another query to complete. This shows that PostgreSQL makes use of locks, and its scheduler is an active locking scheduler: indeed, transaction 2 is actually put in a queue waiting for transaction 1 to release the exclusive lock on the 'Filippo' tuple, set by the scheduler on behalf of transaction 1. This also shows that, since PostgreSQL adopts READ COMMITTED as the lower isolation level, the “reading temporary data” anomaly is prevented.

### *Transaction 1*

We now execute the following actions of transaction 1:

```
END;
```

and the value of "salary" for Filippo in the database is now 20, as can be seen by issuing a SELECT query from another transaction, say transaction 3.

### *Transaction 2*

We go back to transaction 2, and we realize that it proceeded by executing the suspended update. Indeed, if we execute within transaction 2

```
select salary from person where name = 'Filippo';
```

we see the output 30.

We now END transaction 2, thus committing its operations. If we execute

```
select salary from person where name = 'Filippo';
```

from another transaction, we correctly see the output 30.

### 1.3 Scenario 3: locking queues

Assume the initial situation of 26 as the value of the attribute “salary” for Filippo. We consider transaction 1 aiming at assigning 20 to the “salary” of Filippo, transaction 2 aiming at assigning 30 to the “salary” of Filippo and transaction 2 aiming at assigning 40 to the “salary” of Filippo.

#### *Transaction 1*

We execute the following actions of transaction 1:

```
BEGIN;
```

```
update person set salary = 20 where name = 'Filippo';
```

and we leave the session, thus suspending the transaction.

#### *Transaction 2*

We execute the following actions of transaction 2:

```
BEGIN;
```

```
update person set salary = 30 where name = 'Filippo';
```

and what happens at this point is that the system sends a message saying that it is waiting for another query to complete. Transaction 2 is now waiting for transaction 1 to release the exclusive lock on the 'Filippo' tuple, set by the scheduler on behalf of transaction 1.

#### *Transaction 3*

We execute the following actions of transaction 3:

```
BEGIN;
```

```
update person set salary = 40 where name = 'Filippo';
```

and what happens at this point is that the system sends a message saying that it is waiting for another query to complete. Indeed, transaction 3 is actually added to the queue waiting for the exclusive lock on the 'Filippo' tuple to be released.

#### *Transaction 1*

We now execute the following actions of transaction 1:

```
END;
```

and the value of "salary" for Filippo in the database is now 20, as can be seen by issuing a SELECT query from another transaction, say transaction 4.

#### *Transaction 2*

We go back to transaction 2, and we realize that it proceeded by executing the suspended update. Indeed, if we execute within transaction 2

```
select salary from person where name = 'Filippo';
```

we see the output 30.

In the meantime, transaction 3 is still waiting. We now END transaction 2, thus committing its operations. If we execute

```
select salary from person where name = 'Filippo';
```

from another transaction, we correctly see the output 30.

#### *Transaction 3*

We go back to transaction 3, and we realize that it proceeded by executing the suspended update. Indeed, if we execute within transaction 3

```
select salary from person where name = 'Filippo';
```

we see the output 40.

We now END transaction 3, thus committing its operations. If we execute

```
select salary from person where name = 'Filippo';
```

from another transaction, we correctly see the output 40.

## 1.4 Scenario 4: deadlock management

Assume the initial situation of 20 as value of the attribute “age” for Anna, and 80 as value of the attribute “age” for Filippo. We consider transaction 1 aiming at increasing the “age” of both Filippo and Anna, and transaction 2 aiming at decreasing them.

### *Transaction 1*

We execute the following actions of transaction 1:

```
BEGIN;
```

```
update person set age = 81 where name = 'Filippo';
```

and we leave the session and therefore the transaction is suspended.

### *Transaction 2*

We execute the following actions of transaction 2:

```
BEGIN;
```

```
update person set age = 19 where name = 'Anna';
```

and we leave the session and therefore the transaction is suspended.

### *Transaction 1*

We issue the following commands in transaction 1:

```
update person set age = 21 where name = 'Anna';
```

and the system sends a message saying that is waiting for a query to complete. Indeed, this shows that transaction 1 is actually put in a queue waiting for transaction 2 releasing the exclusive lock on the tuple relative to Anna.

### *Transaction 2*

We issue the following commands in transaction 2:

```
update person set age = 79 where name = 'Filippo';
```

and what happens at this point is that the system aborts transaction 2 and sends the message:

```
ERROR: deadlock detected DETAIL: Process 27605 waits for ShareLock on
transaction 1942; blocked by process 24961. Process 24961 waits for ShareLock
on transaction 1943; blocked by process 27605. HINT: See server log for query
details. SQL state: 40P01
```

and if we end transaction 2 by the command:

```
END
```

the system sends the message:

```
ROLLBACK
```

If we go back to transaction 1, we realize that transaction 1 has proceeded, and this is because, once transaction 2 was aborted, it obtained the exclusive lock on the tuple relative to Anna. We can now end transaction 1 by the command:

```
END
```

The database has now the values 81 and 21 for the attribute “age” for Filippo and Anna, respectively.

This shows that the system adopts a recognition approach to deadlock. Also, this example shows that, since PostgreSQL adopts READ COMMITTED as the lower isolation level, the “ghost update” anomaly illustrated in the slide is prevented.

## 2. Level of isolation: READ COMMITTED

We now illustrate several scenarios with the goal of going into more details on the isolation level READ COMMITTED. As we pointed out already, this isolation level is the default in PostgreSQL, in the sense that, even if the isolation level is READ UNCOMMITTED, the system guarantees the behavior of the READ COMMITTED isolation level established by the SQL standard.

Note that when a transaction uses the READ COMMITTED isolation level, there is a difference between the behavior of a transaction constituted by a single SQL statement, in particular, SELECT) within a user-defined transaction T and the whole transaction T:

- A transaction constituted by a single SELECT statement within a user-defined transaction T sees all the changes done by T, and only data committed by other transactions before the query began; it never sees either data changed by uncommitted transactions different from T or changes committed during the SELECT statement execution by concurrent transactions. Actually, a transaction constituted by a single SELECT statement sees a snapshot of the database as of the instant the SELECT statement begins to run.
- However, this is not true for a whole transaction T: it can see changes done by another transaction T' that committed during the execution of T, as the scenarios 5 and 6 illustrated in subsections 2.2 and 2.3 show.

### 2.1 Scenario 5: the use of multi-version concurrency control

Assume the initial situation of 80 as the value for the “age” attribute for Filippo. We will open two sessions, each one with one transaction.

#### *Transaction 1*

We open a session (session 1) and we start executing the following actions of transaction 1:

```
BEGIN;
```

```
update person set age = 50 where name = 'Filippo';
```

and we leave the session (but we do not close it); therefore, the transaction is suspended.

#### *Transaction 2*

We open another session (session 2) and execute the following “select”, corresponding to transaction 2:

```
select * from person where name = 'Filippo';
```

and the output shows the value 80 for the “age” attribute, which is the value stored in the database before the begin of transaction 1.

Note that this means that transaction 1 is working on a local copy (snapshot) of the database. Indeed, the write action of transaction 1, which is still active, is not yet visible in session 2 (transaction 2). In other words, transaction 2 sees a version of the Filippo tuple that is different from the one written by transaction 1. We can now go back to the session of transaction 1 and issue the command:

```
END;
```

If we now re-execute the same instruction of transaction 2 as a new transaction, say transaction 3, we will see 50 as value of “age”, because transaction 1 has committed (by means of the “END” command), and its effects are now visible to the rest of the transactions. So, as expected, transaction 3 sees the current value of the database, while transaction 2 has read a different version of the Filippo tuple with respect to the one currently in the database.

## 2.2 Scenario 6: occurrence of the unrepeatable read anomaly

Assume the initial situation of 80 as the value for the “age” attribute for Filippo. We will open two sessions, each one with one transaction.

### *Transaction 1*

We execute the following actions of transaction 1,

```
BEGIN;  
select * from person where name = 'Filippo';
```

and the output shows the value 80 for the “age” attribute, which is the value stored in the database before the begin of the transaction. We leave the session and therefore the transaction is suspended.

### *Transaction 2*

We execute the following actions of transaction 2, ending it with an implicit COMMIT:

```
BEGIN;  
update person set age = 50 where name = 'Filippo';  
END;
```

### *Transaction 1*

We execute the following action of transaction 1

```
select * from person where name = 'Filippo';
```

and the output shows the value 50 for the “age” attribute, which is the value stored in the database by transaction 2. We then end transaction 1 by END;

Note that this means that the same transaction (in this case transaction 1) can see different values of the same element in different times. This is a case of the anomaly “unrepeatable read”, which is clearly not prevented by the isolation level READ COMMITTED.

## 2.3 Scenario 7: occurrence of the update loss anomaly

Assume the initial situation of 80 as value of the attribute “age” for Filippo. We consider transaction 1 aiming at increasing the “age” of Filippo, and transaction 2 aiming at decreasing it.



#### *Transaction 1*

We execute the following actions of transaction 1:

```
BEGIN;
```

```
update person set age = age + 1 where name = 'Filippo';
```

and we leave the session and therefore the transaction is suspended.

#### *Transaction 2*

We execute the following actions of transaction 2:

```
BEGIN;
```

```
select * from person where name = 'Filippo';
```

and the output shows the value 80 for the “age” attribute.

#### *Transaction 1*

We execute the following actions of transaction 1:

```
END;
```

and the value of age for Filippo in the database is now 81, as can be seen by issuing a SELECT query from another transaction, say transaction 3.

#### *Transaction 2*

We then execute the following actions of transaction 2:

```
update person set age = age - 1 where name = 'Filippo';
```

```
select age from person where name = 'Filippo';
```

producing an output that shows that the age of Filippo is 80, and this value remains after committing transaction 2. The reason is that transaction 1 has committed and therefore transaction 2 now sees the database updated with the actions of transaction 1. Since transaction 1 has increased the age of Filippo by 1 and transaction 2 has decreased such value by one, the result is that the value of the age of Filippo is still 80. In other words, from the point of view of transaction 2, the value of the age of Filippo is exactly the one at the beginning of its execution, and therefore its own update has been lost.

This shows that, with the READ COMMITTED isolation level, the use of locking does not prevent the update loss problem.

## **2.4 Scenario 8: occurrence of the phantom read anomaly**

Assume that the initial situation of the table person is such that the tuples satisfying the condition “age = 20” are as follows:

|          |    |    |
|----------|----|----|
| "Franco" | 20 | 60 |
| "Anna"   | 20 | 50 |

We will open two sessions, each one with one transaction.

#### *Transaction 1*

We execute the following actions of transaction 1

```
BEGIN;
```

```
select * from person where age = 20;
```

and the output shows:

|          |    |    |
|----------|----|----|
| "Franco" | 20 | 60 |
| "Anna"   | 20 | 50 |

#### *Transaction 2*

We execute the following actions of transaction 2

```
BEGIN;  
insert into person values ('Ugo', 20, 40);  
END;
```

#### *Transaction 1*

We execute the following actions of transaction 1

```
select * from person where age = 20;
```

and the output shows:

|          |    |    |
|----------|----|----|
| "Franco" | 20 | 60 |
| "Anna"   | 20 | 50 |
| "Ugo"    | 20 | 40 |

This means that, in Postgres, the phantom read anomaly is not prevented in the isolation level “READ COMMITTED”.

### **3. Level of isolation: REPEATABLE READ**

We now analyze the level of isolation REPEATABLE READ. We remind the reader that in order to enforce this level of isolation in a certain transaction we have to issue the command

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

#### **3.1 Scenario 9: avoiding the unrepeatable read anomaly**

Assume the initial situation of 80 as the value for the “age” attribute for Filippo. We will open two sessions, each one with one transaction.

#### *Transaction 1*

We execute the following actions of transaction 1,

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
select * from person where name = 'Filippo';
```

and the output shows the value 80 for the “age” attribute, which is the value stored in the database before the begin of the transaction. We leave the session and therefore the transaction is suspended. Note that the transaction is defined with the isolation level “repeatable read”.

### *Transaction 2*

We execute the following actions of transaction 2:

```
BEGIN;  
update person set age = 50 where name = 'Filippo';  
END;
```

### *Transaction 1*

We execute the following actions of transaction 1

```
select * from person where name = 'Filippo';
```

and the output shows the value 80 for the “age” attribute, which is the value initially stored in the database when transaction 1 started. We then end the transaction by

```
END;
```

Note that this means that the REPEATABLE READ mode starts each transaction with a new snapshot that includes the effects of all transactions committed up to that moment. Thus, with REPEATABLE READ, the level of isolation increases. The two read operations within the same transaction 1 see the same data.

Obviously, if we now execute a new transaction consisting of the instruction of transaction 1 again, then we would see the value 50 for the “age” attribute for Filippo.

## **3.2 Scenario 10: avoiding the update loss anomaly**

Assume the initial situation of 80 as value of the attribute “age” for Filippo. We consider transaction 1 aiming at increasing the “age” of Filippo, and transaction 2 aiming at decreasing it.

### *Transaction 1*

We execute the following actions of transaction 1:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
update person set age = age + 1 where name = 'Filippo';
```

and we leave the session and therefore the transaction is suspended.

### *Transaction 2*

We execute the following actions of transaction 2:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
update person set age = age - 1 where name = 'Filippo';
```

and what happens at this point is that the system sends a message saying that is waiting for another query to complete. Indeed, transaction 2 is actually put in a queue waiting for transaction 1 to release the exclusive lock, set by the scheduler on behalf of transaction 1, on the tuple relative to Filippo.

### *Transaction 1*

We execute the following actions of transaction 1:

```
END;
```

and the system sends a message saying that the transaction committed.

#### *Transaction 2*

If we go back to the session with transaction 2, we see the following message sent by the system:

```
ERROR: could not serialize access due to concurrent update SQL state:
40001
```

and we realize that transaction 2 is aborted. This is because with the isolation level repeatable read, no transaction T can work on data modified by another transaction committed after T started. This implies that, in Postgres, the update loss anomaly is prevented in the isolation level REPEATABLE READ.

Note that, if we had ended the transaction 1 with the command

```
ROLLBACK;
```

then transaction 2 would have resumed and the value of the “age” attribute for Filippo would have been modified to 79.

### **3.3 Scenario 11: avoiding the phantom read anomaly**

Assume that the initial situation of the table person is such that the tuples satisfying the condition “age = 20” are as follows:

|          |    |    |
|----------|----|----|
| "Franco" | 20 | 60 |
| "Anna"   | 20 | 50 |

We will open two sessions, each one with one transaction.

#### *Transaction 1*

We execute the following actions of transaction 1

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
select * from person where age = 20;
```

and the output shows:

|          |    |    |
|----------|----|----|
| "Franco" | 20 | 60 |
| "Anna"   | 20 | 50 |

#### *Transaction 2*

We execute the following actions of transaction 2

```
BEGIN;
insert into person values ('Ugo', 20, 40);
END;
```

#### *Transaction 1*

We execute the following actions of transaction 1

```
select * from person where age = 20;
```

and the output shows:

|          |    |    |
|----------|----|----|
| "Franco" | 20 | 60 |
| "Anna"   | 20 | 50 |

This means that, in Postgres, the phantom read anomaly is prevented in the isolation level “REPEATABLE READ”. This is not surprising, since, after all, the phantom read anomaly is a kind of “unrepeatable read anomaly” at the level of a set of tuples, rather than just one tuple. Note that the described behavior seems in contrast to the SQL standard, but it is not, because, as we said before, the SQL standard allows implementing a stronger version of a certain isolation level with respect to what the standard specifies.

## 4. Level of isolation: SERIALIZABLE

We now consider a schema built by:

```
CREATE TABLE mytab
(
    class integer,
    value integer
);
```

with rows:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |

and consider two processes P1 and P2, where one aims at executing

insert into mytab (select 2, sum(value) from mytab where class = 1);

and the other one aims at executing

insert into mytab (select 1, sum(value) from mytab where class = 2);

Note that if we execute them in serial order, we have the following alternative results. If the order is <P1,P2>, then the result is R1:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |
| 2, | 30  |
| 1, | 330 |

If the order is <P2,P1>, then the result is R2:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |
| 1, | 300 |
| 2, | 330 |

Now, consider the following possible concurrent execution of the two processes, expressed by two transactions operating under the REPEATABLE READ isolation level.

*Transaction 1:*

We execute the following actions of transaction 1:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
insert into mytab
    (select 2, sum(value) from mytab where class = 1);
```

*Transaction 2:*

We execute the following actions of transaction 2:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
insert into mytab
    (select 1, sum(value) from mytab where class = 2);
END
```

Right now, if you open a new transaction and we inspect that table mytab, you realize that transaction 2 has committed and the tuples of mytab the database are:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |
| 1, | 300 |

We now execute the following action in transaction 1:

```
select * from mytab
```

obtaining:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |
| 2, | 30  |

Note that transaction 1 is still active, and the effect of transaction 2 is not visible in transaction 1, according to the isolation level REPEATABLE READ. But now we END transaction 1, and if we ask in a new transaction for all the tuples of mytab, we obtain:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |
| 2, | 30  |
| 1, | 300 |

Since we get neither result R1, nor result R2, we conclude that the system has allowed a non-serializable behavior.

Let us now define and execute the same concurrent processes, expressed by two transactions operating under the SERIALIZABLE level.

*Transaction 1:*

We execute the following actions of transaction 1:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
insert into mytab
    (select 2, sum(value) from mytab where class = 1);
```

*Transaction 2:*

We execute the following actions of transaction 2:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
insert into mytab
    (select 1, sum(value) from mytab where class = 2);
END
```

and we get the message:

COMMIT

Right now, if you open a new transaction and we inspect that table mytab, you realize that transaction 2 has committed and the tuples of mytab the database are:

|    |     |
|----|-----|
| 1, | 10  |
| 1, | 20  |
| 2, | 100 |
| 2, | 200 |
| 1, | 300 |

*Transaction 1:*

We go back to transaction 1 and execute

```
select * from mytab
```

and obtain the following message:

ERROR: could not serialize access due to read/write dependencies among transactions DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt. HINT: The transaction might succeed if retried. SQL state: 40001

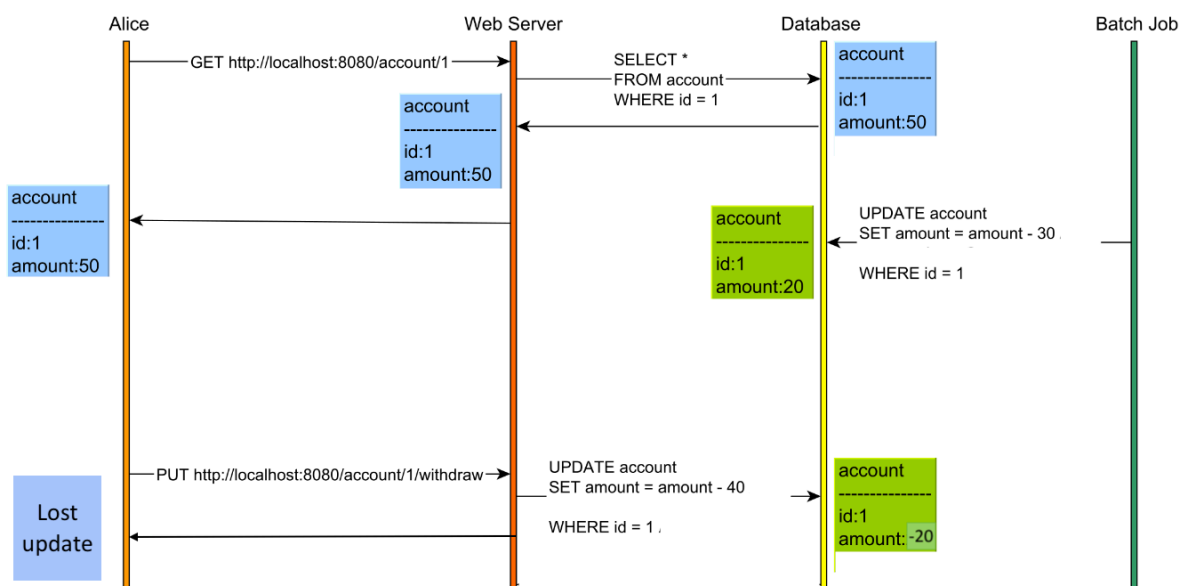
This reflects the fact that the schedule in input is not serializable and the system has raised an error for transaction 1, that must now be rolled back.

## 5. Considerations on the three-tier architecture

The SERIALIZABLE isolation level ensures an adequate treatment of concurrency guaranteed by the data server. This implies that in a client-server architecture where the data server is implemented, for example, by a DBMS like PostgreSQL, concurrency control can rely on a sound set of methods.

Notice, however, that in three-tier architecture, things are more complicated. Indeed, although the interaction between the application layer and the data layer can be based on the same principles as discussed above, the presence of the presentation layer introduces several issues.

Consider, for example, the situation illustrated in the figure below, where we assume that the database (the data layer) is managed by a DBMS functioning with the SERIALIZABLE isolation level, and the client Alice interacts with a Web Server (at the application layer).



Alice issues a GET command in order to know the amount of money in the account with id=1. The Web Server issues a transaction consisting of the appropriate SELECT statement, which



informs Alice that the account has 50 Euros. After such transaction, another DMBS user (batch job) updates the same account by means of a transaction consisting of the appropriate UPDATE statement, withdrawing 20 euros. At this point, Alice decides that she can withdraw 40 euros, based on the information she got from the database (50 euros in her account), and she issues the PUT command and the Web Server executes the transaction consisting of the appropriate UPDATE statement. At the end of this operation, the account has -20 as a balance!

It is not difficult to realize that, from the point of view of Alice, the interaction suffers from a severe concurrency anomaly, caused by an unsatisfactory behavior of the application level.

The example confirms what we already said during our study of concurrency control: it is the responsibility of the application level to use the DBMS correctly, by presenting the right set of transactions, depending on the logic of the application.

In the example above, it is clear that the problem could and should be solved by the application level, in particular by adopting suitable strategy for dealing with concurrency issues involving its clients. One possibility is, for example, to put all the commands by Alice in a transaction. Another possibility is to refuse the update of Alice, informing her that the amount of money changed after the last query.

We can thus conclude that all the notions, principles and techniques we have studied in the context of concurrency control within DBMSs constitute an important set of conceptual and technical tools that may help in designing correct applications.